

```

class Solution {
    public int[][] matrixBlockSum(int[][] mat, int k) {
        int m = mat.length, n = mat[0].length;
        int[][] prefix = new int[m + 1][n + 1];

        // Build prefix sum matrix
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                prefix[i + 1][j + 1] = mat[i][j]
                    + prefix[i][j + 1]
                    + prefix[i + 1][j]
                    - prefix[i][j];
            }
        }

        int[][] ans = new int[m][n];

        // Compute block sum using prefix sums
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                int r1 = Math.max(0, i - k), c1 = Math.max(0, j - k);
                int r2 = Math.min(m - 1, i + k), c2 = Math.min(n - 1, j + k);

                ans[i][j] = prefix[r2 + 1][c2 + 1]
                    - prefix[r1][c2 + 1]
                    - prefix[r2 + 1][c1]
                    + prefix[r1][c1];
            }
        }

        return ans;
    }
}

```

Accepted 12 / 12 testcases passed

Mahipal_Reddy_VTU25525 submitted at Feb 11, 2026 10:10

[Solution](#)

Runtime

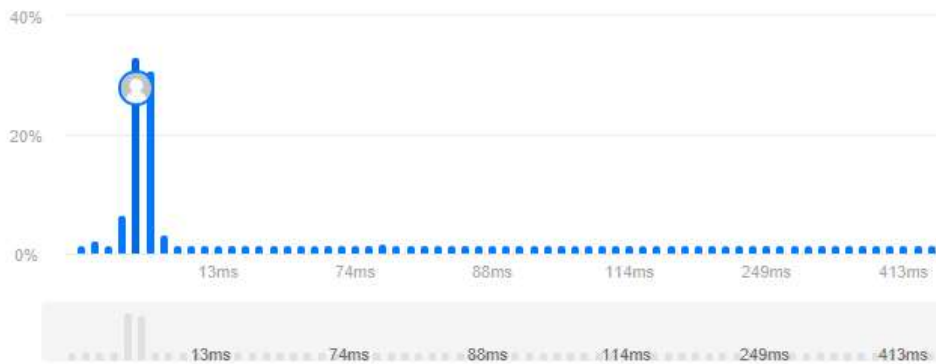


6 ms | Beats 89.61%

[Analyze Complexity](#)

Memory

47.13 MB | Beats 8.85%



Code | Java

```
1 class Solution {
2     public int[][] matrixBlockSum(int[][] mat, int k) {
3         int m = mat.length, n = mat[0].length;
4         int[][] prefix = new int[m + 1][n + 1];
5
6         // Build prefix sum matrix
7         for (int i = 0; i < m; i++) {
8             for (int j = 0; j < n; j++) {
```

[View more](#)